

Développons Nos compétences dans :

- ✓ Fonctions et procédures Stockées
- ✓ Triggers

Introduction : Dans ce TP nous allons mettre en œuvre les fonctions et procédures stockées pour bien appréhender leurs rôles dans un SGBD.

Partie I- Les Procédures Stockées

1- Création et exécution d'une procédure stockée

Afin de créer une procédure stockée vous placer l'ordre suivant :

CREATE PROCEDURE salaire_moyen is

Avec le bloc de programme qui calcule la moyenne des salaires de la table EMPLOYEES

```
DECLARE
moye NUMBER(7,2);
Begin
SELECT AVG(salary) INTO moye FROM EMPLOYEES;
DBMS_OUTPUT.PUT_LINE ('à la date du ' || SYSDATE);
DBMS_OUTPUT.PUT_LINE ('La moyenne des salaires est : ' || moye);
DBMS_OUTPUT.PUT_LINE ('Préparer par ' || USER);
END;
```

1.1 Écrire et stocker ce code tant que procédure.

1.2 Compiler la procédure

1.3 Si c'est bien compiler exécuter la procédure avec les 3 méthodes qu'on a vu au cours.

2- procédure stockée avec paramètres

2.1 Écrire une procédure sockets qui prend en paramètre le **department_id** et renvoie dans un paramètre la moyenne des salaires des employés de ce département

NB : la procédure n'affiche rien à la sortie

2.2 Écrire un bloc PL/SQL où vous faites appel à la procédure pour afficher la moyenne des salaires de 3 départements de votre choix

3- Autre Procédure

Écrire un bloc PL / SQL pour définir la procédure **employees_details** qui prend en entrée un numéro de l'employé **employee_id** et renvoie le détail suivante **employee_id**, **nom** et **salaire**, envisagez une exception au cas où le **employee_id** n'existe pas dans la table **EMPLOYEES**. **NB** : cette procédure n'affiche rien à la sorti

3.1 Appeler dans un bloc PL/SQL la procédure **employees_details** avec l'employée_id 102 et affiche à la sortie le détail avec le salaire augmenté de 10%.

3.2 Faire appel à la procédure **employees_details** pour afficher le détail pour tous les employés de la table EMPLOYEES

4- Déclarer et appeler des procédures au sein de procédures

4.1 Étudier le code ci-dessous(**ne le taper pour le moment**) notez tout d'abord la sortie que vous attendez du programme ?

4.2 Maintenant taper ce code, puis exécuter la procédure principale et voir si vous obtenez la même chose que vous avez notée.

```
create or replace procedure proc_principale
is
  v_test number:=4;
  procedure proc_intern_1
  is
  begin
    dbms_output.put_line(v_test || ' la procedure interne 1');
  end proc_intern_1;
  procedure proc_intern_2
  is
  begin
    dbms_output.put_line(v_test || ' la procedure interne 2');
  end proc_intern_2;
  begin
    while v_test <=10 loop
      proc_intern_1;
      if mod(v_test,2)=0 then
        proc_intern_2;
      end if;
      v_test:=v_test+1;
    end loop;
  end proc_principale;
```

Partie II- Les Fonctions Stockées

1- Écrire une fonction **nmb_emp_job** qui accepte le JOB_ID comme paramètre et renvoie le nombre des employés qui occupent ce JOB.

1.1 Faites des exemples d'exécutions pour s'assurer de la validité de votre fonction

Entre autre vous faites **SELECT nmb_emp_job (AD_PRES) from dual;**

1.2 Faites appel à cette fonction dans une procédure stockée pour recenser le nombre d'employés par JOB.

Après avoir exécuté la procédure sockée vous aurez le message suivant :

Dans la société on trouve : 15 Administration Assistant 20 Accountant 7 Sales Manager ...ect
--

- 2- Étudier le code ci-dessous qui utilise des fonctions intégrées dans une fonction principale pour faire un peu d'arithmétique assez trivial.

Portez une attention particulière au retour (RETURN).

```
create or replace function calculate
(v_1 in number, op in varchar2, v_2 in number)
return number
is
v_return number;
function add
return number
is
begin
return v_1+v_2;
end add;
function subtract
return number
is
begin
return v_1-v_2;
end subtract;
function multiply
return number
is
begin
return v_1*v_2;
end multiply;
function divide
return number
is
begin
return v_1/v_2;
exception when zero_divide
return -1;
end divide;
begin
if op='+' then v_return:=Add;
elsif op='-' then v_return:=subtract;
elsif op='*' then v_return:=multiply;
elsif op='/' then v_return:=divide;
```

```
else v_return:=null;
end if;
return v_return;
end calculate;
```

2.1 Pour tester le programme, vous pouvez utiliser la table fictive DUAL de Oracle dans les instructions SELECT. Par exemple, essayez:

Select SELECT (2, '+', 3) from dual ;

2.2 Cela devrait appeler la fonction pour effectuer l'arithmétique demandée. Assurez-vous que vous comprenez comment l' «Appel» est réalisé dans la déclaration ci-dessus.

2.3 Essayez de passer différents paramètres pour tester si le code fonctionne correctement.

2.4 Essayez de passer un zéro pour une division par exemple! Proposer une solution pour ce cas ?

3- Autre fonctions

3.1 Écrire une fonction **f_prime** qui calcule la prime en fonction de salaire, elle admet deux paramètres le salaire (salary) et la commission (COMMISSION_PCT) et retourne la prime

- ✓ La formule est : Prime= salaire * commission.
- ✓ Prévoir le cas où la commission est nulle (utiliser **NVL**)

3.2 Écrire une fonction **f_taxe** qui calcule l'impôt sur le salaire d'un employé. La fonction **f_taxe** admet deux paramètres qui sont le **salaire** et la **prime**.

Ci-dessous les formules et les informations à respecter lors de la réalisation de la fonction :

- Passer deux paramètres de salaire et de prime

Le salaire imposable est le salaire + bonus - abattement fiscal

Abattement fiscal est une constante égale à 4335 \$

- Il y a deux limites fiscales à 2500 \$ (inférieure) et 20 000 \$ (supérieure)

Le Salaire est donc à tester face à ces limites fiscales

- ✓ Si le salaire est inférieur à la limite inférieure d'impôt alors taxe est calculée à 10% du salaire imposable
- ✓ Si le salaire est entre les deux limites fiscales alors l'impôt est calculé comme 20% du salaire imposable
- ✓ Si le salaire est plus élevé que la limite supérieure de l'impôt alors taxe est calculée comme suit:

20% de la limite supérieure de + 38% de la différence entre le salaire imposable et la limite supérieure.

3.3 Vous devriez alors être en mesure de tester votre fonction en tapant la requête ci-dessous

```
SELECT first_name, last_name, salary, f_taxe (salary, f_prime(salary, COMMISSION_PCT)
from EMPLOYEES ;
```

Partie III- Les Triggers

Introduction

Par le biais de cette partie de TP, nous allons apprendre à créer les triggers, les manipuler avec des jeux de tests afin de les assimiler davantage et d'appréhender leur rôle dans un SGBD.

Exercice 1

La table **ARTICLES** (nom_Prod (varchar2), Quantite (number), Prix_u(number), cout (number)) contient les détails sur les produits, où **cout** est une colonne calculée, elle enregistre le cout de l'article, elle est calculée par $\rightarrow \text{cout} = \text{prix_u} * \text{quantite}$

- 1- Créer cette table
- 2- Insérer la ligne suivante nom_prod= LapTopHP, Quantite=5, prix=3600,00
- 3- Faire un **select* from articles**, Que est ce que vous constaterez pour le champ **cout** ?
- 4- Vous allez remédier à ce constat en créant le trigger suivant

```
CREATE OR REPLACE TRIGGER ARTCILE_TRG1
BEFORE
INSERT OR UPDATE ON ARTICLES
FOR EACH ROW
BEGIN
:NEW.COUT := :NEW.QUANTITE * :NEW.PRIX;
END;
```

5- Test de Trigger **ARTCILE_TRG1**

5.1 - Maintenant insérer les lignes suivantes dans la table ARTICLES

Nom_prod	quantite	Prix
Lap_Top_COM	10	2000,5
Mouse_king	40	20,00
USB_Key_Row	60	23,00
Lap_Top_Del	2	2300,00

5.2 - faire un **select * from ARTICLES**, Que est ce que vous constaterez pour le champ **cout** ? Expliquer ... ?

5.3 – en une seule instruction, Procéder à une mise à jour du prix de tous les articles

Lap_Top à 2500, Que est ce que vous constaterez pour le champ **cout** ? Expliquer ... ?

6- Le statut de trigger peut être activé ou désactivé (ENABLE or DISABLE)

6.1 Vérifier que le trigger **ARTCILE_TRG1** est actif, en consultant le **dictionnaire de données** par la requête suivante:

- **select status from all_triggers where trigger_name like 'ARTCILE_TRG1';**

6.2 On peut gérer les trigger par la commande **ALTER TRIGGER**

Par exemple, pour désactiver le trigger **ARTCILE_TRG1** :

```
ALTER TRIGGER ARTCILE_TRG1 DISABLE
```

Et pour l'activer

```
ALTER TRIGGER ARTCILE_TRG1 ENABLE
```

- a- Désactiver le trigger et essayer d'insérer un nouveau enregistrement ou de MAJ la table **ARTCILES**, Que est ce que vous constater pour le champ **cout** ? Expliquer ... ?
- b- Activer le de nouveau et essayer d'insérer un nouveau enregistrement ou de MAJ la table **ARTCILES**, Que est ce que vous constater pour le champ **cout** ? Expliquer ... ?

Exercice 2

Les déclencheurs (Triggers) peuvent **déclarer des exceptions** et prévenir une modification indésirable d'une table. Dans cet exemple nous allons créer un trigger **EMP_TRG1** qui bloque une modification du salaire d'un employée dans la table **EMPLOYEES** parvenu d'un utilisateur non autorisé à le faire

```
CREATE OR REPLACE TRIGGER EMP_TRG1
BEFORE UPDATE OF SALARY ON EMPLOYEES
FOR EACH ROW
DECLARE
    User_non_autorise EXCEPTION;
BEGIN
    IF (USER <> 'HR') THEN
        RAISE User_non_autorise ;
    END IF;
    EXCEPTION WHEN User_non_autorise THEN
        RAISE_APPLICATION_ERROR(-20001, 'SEUL le MANAGER peut
        CHANGER le Salaire');
    END;
```

Le trigger utilise la procédure **RAISE_APPLICATION_ERROR**, qui prend deux paramètres, **Numéro de l'erreur** (tout numéro entre -20001 et -20999) et le **message d'erreur** qu'on désire afficher à l'écran.

1- Pour tester ce trigger

1.a) faire un update de salaire d'un employé de votre choix

1.b) créer un autre utilisateur **TEST_ESTM**

```
Create user TEST_ESTM identified by 'motpass' ;
Grant resource,connect to TEST_ESTM ;
Grant update on EMPLOYEES to TEST_ESTM ;
```

1.c) se connecter en tant que **TEST_ESTM** et faire la MAJ de salaire des employées de département 30. Qu'est ce que vous constatez ?, expliquez ?

- 2- Désactiver le trigger et faire le même 'UPDATE que dans 1.c par l'utilisateur **TEST_ESTM** .
Qu'est ce que vous constatez ?, expliquez ?

Exercice 3

Créer la table **EVALUATION** (id , note_DS1,note_DS2,note_DSTP, Moyenne) tout ces champ sont de type number , la colonne moyenne est la moyenne des notes des 3 DS

- 1- On veut créer un trigger Evalu_TRG pour calculer la moyenne quelles sont les order MLD à prévoir pour ce trigger
- 2- Créer le trigger Evalu_TRG
- 3- Tester ce trigger avec les order MLD concernées

Exercice 4

La table ARTICLES créée dans l'exercice 1 est objet à la règle de gestion suivante

Le prix de l'article ne peut pas être changé de moins au plus de 15%.

- 1- Créer le trigger ARTICLES_TRIG2 qui mit en œuvre cette règle
- 2- Faites des tests en changeant les prix de quelque produit.

Exercice 5

On travaille sur le schéma relationnel 3FN suivant (les attributs soulignés sont les clés de la relation) :

EMPLOYE (NUMEMP,NOM,NUMSERV,NUMCHEF,SALAIRE)

PROJET (NUMPROJ,NOMPROJ, NUMRESPONSABLE)

PARTICIPE (NUMEMP, NUMPROJ, QUOTA) → Quota : c'est le **pourcentage** de participation de l'employer(NUMEMP) dans le projet (NUMPROJ)

Créer les triggers permettant de :

- a- Refuser de nommer responsable d'un projet quelqu'un qui n'y participe pas.
- b- Modifier le trigger, en mettant l'employé sujet de la nomination comme participant à ce projet, s'il n'y participe pas.
- 1- Refuser de supprimer un employé de la table participant si cet employé est responsable de projet
- 2- Vérifier que la somme des quotas de participation d'un employé aux différents projets ne dépasse pas 100. (afficher un message d'erreur en arrêtant l'opération à l'aide de **raise_application_error**).